

ITC MP, DS n°2

Attention, le sujet est très long (longueur de sujet de concours globalement). Il est en très grande partie extrait du sujet des mines 2024.

N'hésitez pas à sauter des questions pour faire celles que vous préférez.

Rappels concernant le langage Python : il n'est pas possible d'utiliser des fonctions internes à Python sur les listes ou les chaînes de caractères telles que `min`, `max`, `count`, `remove` ... Seules les instructions basiques telles que `len(liste)`, `liste.append(e)` sont autorisées. Le tranchage ou slicing ainsi que la concaténation sont également permis. Les programmes doivent être commentés lorsque c'est nécessaire pour justifier les choix.

Lorsqu'on écrit, dans le sujet,

```
def fonction(arg1: int, arg2: list) -> float:
    ...
    ...
    ...
```

Cela sert à préciser le type des arguments et de la valeur de retour de la fonction. Ici, la fonction prend en argument un entier et une liste et renvoie un flottant.

Il n'est pas nécessaire de recopier ou de préciser ces informations dans vos fonctions.

Introduction à deux problèmes en communication numérique

Introduction

On s'intéresse au problème de communication entre deux personnes, nommées Alice et Bob qui cherchent à s'envoyer un message au travers d'un canal de communication (une bande de fréquences radio par exemple).

Avant d'être lu par Bob, le message original d'Alice passe par plusieurs étapes que nous allons séparer de la manière suivante :

- une phase de compression, durant laquelle Alice cherche à trouver la représentation la plus compacte possible du message,
- une phase d'encodage durant laquelle le message compressé est transformé en une succession de symboles transmissibles au travers du canal de communication utilisé,
- une phase de transmission durant laquelle le message encodé circule sur le canal de communication et est susceptible de subir une altération,
- une phase de décodage durant laquelle Bob décode le message qu'il a reçu, le message lui apparaît alors sous la forme compressée,
- une phase de décompression durant laquelle Bob applique l'opération réciproque de la compression opérée par Alice.

Ce modèle est décrit par le schéma de la Figure 1 :

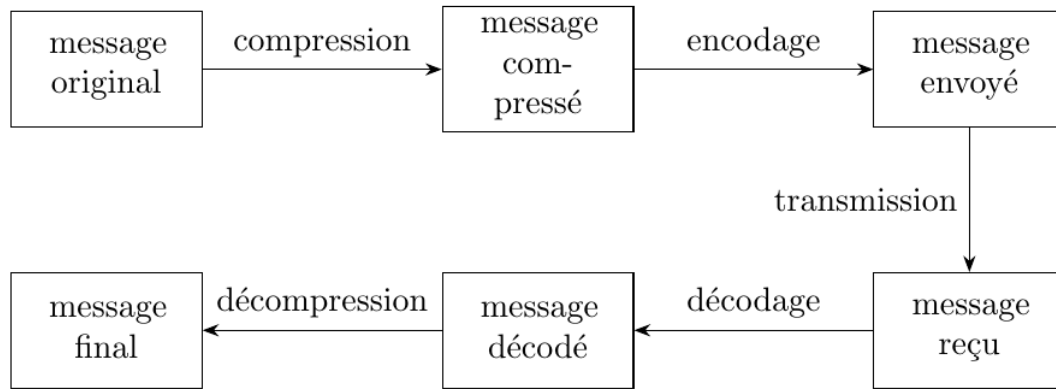


Figure 1: Schématisation du modèle de communication considéré ici.

I) Compression du message d'Alice : codage arithmétique

Le codage ASCII (American Standard Code for Information Interchange) définit une norme où 128 caractères sont codés sur 7 bits. Ce codage est illustré par les quelques lignes suivantes :

Caractère	Codage binaire	Équivalent numérique
'a'	110 0001	97
'b'	110 0010	98
'z'	111 1010	122

Lorsqu'une chaîne de caractères n'utilise pas l'intégralité des 128 caractères proposés par le codage ASCII, il est possible de convenir d'une représentation différente, plus économique en nombre de symboles. Considérons la chaîne de caractères $s = \text{'abaabaca'}$.

On peut proposer de coder celle-ci à l'aide du tableau suivant :

Caractère	Code
'a'	00
'b'	01
'c'	10

Dans ce cas, la chaîne de caractères s est codée sur 16 bits par :

00 01 00 00 01 00 10 00

(où des espaces ont été introduits pour faciliter la lecture).

Dans un souci de compression de l'information, il est intéressant de représenter les caractères les plus fréquents par des expressions courtes et de ne plus nécessairement coder avec des codes de longueur constante chaque caractère. Dans l'exemple précédent, il est possible de coder le caractère 'a' avec 1 bit, et les caractères 'b' et 'c' avec 2 bits afin de coder la chaîne s sur seulement 11 bits en tout.

Q1) Proposer une telle représentation en expliquant pourquoi celle-ci pourra être décodée sans ambiguïté. Vous ferez en sorte que la représentation binaire de 'a' soit inférieure à celle de 'b', elle-même inférieure à celle de 'c'.

Caractère	Code
'a'	1
'b'	01
'c'	00

Si on lis un zéro, on peut lire le caractère immédiatement après pour savoir si c'est un a ou un b. Si on lis un 1 sans avoir lu de zéro juste avant, c'est que c'est un a.

La représentation précédente emploie la même longueur pour coder les caractères 'b' et 'c' alors que le caractère 'b' est deux fois plus présent que le caractère 'c' dans la chaîne s.

Il est possible d'aller un cran plus loin et le codage arithmétique présenté dans cette étude permet un gain de compression comme s'il parvenait à représenter un caractère avec un nombre non entier de bits au prorata de sa fréquence d'apparition. Ce principe de compression est notamment utilisé par la norme JPEG2000 de compression des images. Nous ne le présenterons cependant ici que dans le cadre de l'étude de chaînes de caractères.

I.1) Analyse du texte source

L'objet de cette partie est d'analyser le contenu d'une chaîne de caractères s afin de déterminer :

- les caractères utilisés par la chaîne s ;
- le nombre d'occurrences de chacun.

Indication python On peut accéder aux caractères d'une chaîne de caractère, à sa longueur, ou la parcourir avec une boucle, exactement de la même façon qu'une liste. Il n'est en revanche pas possible de faire .append et .pop. Pour concaténer deux chaînes, on utilise le +.

Q2.) Écrire une fonction nommée nb_caracteres(c:str,s:str)->int qui prend comme argument un caractère c, une chaîne s et qui renvoie le nombre d'occurrences (c'est-à-dire le nombre d'apparitions) de c dans s. La fonction doit avoir une complexité linéaire en n, la longueur de la chaîne s.

```
def nb_caracteres(c, s):
    """Renvoie le nombre d'occurrences de c dans s"""
    cpt = 0
    for x in s:
        if x == c:
            cpt += 1
    return cpt
```

Pour déterminer la liste des caractères utilisés à l'intérieur d'une chaîne s on utilise la fonction définie ci-dessous :

```
def liste_caracteres(s:str):
    liste_car = []
    n = len(s)
    for i in range(n):
        c = s[i]
        if not (c in liste_car):
            liste_car.append(c)
    return liste_car
```

Q3.) Que renvoie cette fonction lorsque s='abaabaca' ? Expliquer succinctement le principe de fonctionnement de cette fonction.

Elle renvoie ['a', 'b', 'c']. Cette fonction parcourt les caractères de la liste, et, quand elle en trouve un qui n'a jamais été vu, elle l'ajoute à `liste_car`.

Q4.) En fonction de la longueur n de la chaîne et du nombre k de caractères distincts dans celle-ci, déterminer la complexité asymptotique dans le pire des cas de la fonction de la question Q3. Par exemple pour $s = \text{'abaabaca'}$, on a $n = 8$ et $k = 3$. On négligera la complexité des `append` mais pas celle des tests d'appartenance de la forme `i in L`.

Autrement dit, la ligne 7 est considérée comme étant de complexité constante et la ligne 6 de complexité linéaire en la longueur de la liste `listeCar`.

La fonction, dans ce cas, est en $O(nk)$

On définit alors une fonction analyseTexte(s:str) ->list

```
def analyse_texte(s:str):
    R = []
    l = liste_caracteres(s)
    for i in range(len(l)):
        c = l[i]
        R.append((c, nb_caracteres(c, s)))
    return R
```

Q5.) Expliquer ce que fait cette fonction et donner la valeur renvoyée par la commande analyse_texte('babaaaabca').

Elle renvoie le nombre d'occurrences de chaque caractère dans le texte.

[('b', 3), ('a', 6), ('c', 1)]

Q6.) En fonction de la longueur n de s et du nombre k de caractères distincts présents dans s , (autrement dit k est la longueur de listeCaracteres(s)), donner une estimation de la complexité asymptotique dans le pire des cas de la fonction analyse_texte.

liste_caracteres est en $O(nk)$, et ensuite on appelle nb_caractere k fois.

Au total, cela donne $O(nk)$.

Q7.) Écrivez une fonction qui permet de trier la liste renvoyée par analyse_texte en fonction du nombre d'occurrences de chaque lettre. Précisez le nom de l'algorithme utilisé et sa complexité.

```
def tri_rapide(L):
    if len(L) <= 1:
        return L[:]
    pivot = L[0]
    l1 = [x if x[1] < pivot[1] for x in L[1:]]
    l2 = [x if x[1] >= pivot[1] for x in L[1:]]
    return tri_rapide(l1) + [pivot] + tri_rapide(l2)
```

Tri rapide, complexité moyenne en $O(n \log n)$, pire cas en $O(n^2)$.

Q8.) Écrivez une fonction plus_proche(liste_occurrences, k) qui prend en argument la liste triée et un entier, et qui permet de trouver le caractère qui a le nombre d'occurrences le plus proche de k . Votre fonction devra avoir une complexité $O(\log(n))$ où n est la longueur de la liste_occurrences.

```
def plus_proche(L, k)
    a = 0
    b = len(L)
    while b-a > 1:
        milieu = (a + b) // 2
        if L[milieu][1] < k:
            a = milieu
        elif L[milieu][1] > k:
            b = milieu
        else:
            return L[milieu][0]
    return L[a][0]
```

Q9.) Adapter la fonction de la question Q5 pour qu'elle utilise (et renvoie) un dictionnaire. Elle devra avoir une complexité,

1. linéaire en la longueur n de s ,
2. indépendante de k nombre de caractères distincts présents dans s .

De plus, cette fonction devra impérativement ne parcourir qu'une seule fois la chaîne de caractères. On admettra qu'un test d'appartenance d'une clé à un dictionnaire se fait à coût constant. Par exemple, `analyseTexte('abracadabra')` renverra `{'a':5, 'b':2, 'r':2, 'c':1, 'd':1}`.

```
def analyse_texte(s):
    occurrences = {}
    for c in s:
        if c not in occurrences:
            occurrences[c] = 0
        occurrences[c] += 1
    return occurrences
```

I.2) Compression

La compression par codage arithmétique consiste à représenter une chaîne de caractères s par un nombre réel déterminé à l'intérieur de l'intervalle $[0, 1[$. Initialement, on attribue à chaque caractère utile une portion de l'intervalle $[0, 1[$ proportionnelle à sa fréquence d'occurrences. Par exemple, pour un alphabet à 5 lettres 'abcde', on pourrait avoir un tableau comme ci-dessous :

Caractère	'a'	'b'	'c'	'd'	'e'
Fréquence	0.2	0.1	0.2	0.4	0.1
Intervalle	$[0, 0.2[$	$[0.2, 0.3[$	$[0.3, 0.5[$	$[0.5, 0.9[$	$[0.9, 1.0[$

La chaîne de caractères s est codée en partant de l'intervalle $[0, 1[$. À chaque caractère successif de celle-ci, on affine cet intervalle en ne considérant que la portion correspondant au caractère lu. Si par exemple la chaîne à coder est $s = \text{'dac'}$:

- on obtient d'abord l'intervalle $[0.5, 0.9[$ correspondant au caractère 'd' ;
- le caractère 'a' détermine alors le sous-intervalle $[0.50, 0.58[$ de $[0.5, 0.9[$ correspondant à la portion associée au caractère 'a'.
- le caractère 'c' détermine enfin l'intervalle $0.524, 0.540$.

La figure qui suit illustre ce processus.

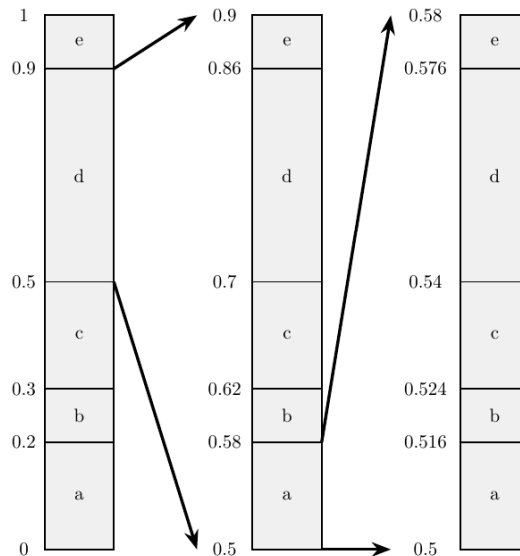


Figure 2: Encodage de $s = \text{'dac'}$

Q10.) En considérant la table des fréquences précédente, proposer l'intervalle correspondant à la chaîne $s = \text{'bac'}$.

$[0.2, 0.3[\rightarrow [0.2, 0.22[\rightarrow [0.206, 0.21[$

On suppose disposer d'une fonction `codeCar(car:str,g:float,d:float)->(float,float)` qui prend en argument un caractère `car` et les deux extrémités d'un intervalle $[g, d[$ et qui renvoie un tuple composé des extrémités du sous-intervalle de $[g, d[$ déterminé par le caractère `car`. En reprenant l'illustration précédente `codeCar('b',0,1)` produit $(0.2, 0.3)$ et `codeCar('a',0.5,0.9)` produit $(0.5, 0.58)$.

Q11.) Écrire une fonction `codage(s:str)->(float,float)` prenant en argument la chaîne s et fournissant en réponse le tuple (g, d) constitué des deux extrémités de l'intervalle $[g, d[$ produit par l'algorithme de codage précédent.

```
def codage(s):
    a = 0
    b = 1
    for c in s:
        a, b = codeCar(c, a, b)
    return a, b
```

Le codage arithmétique consiste alors à coder la chaîne s par un flottant x choisi arbitrairement à l'intérieur de l'intervalle $[g, d[$.

I.3) Décodage

Pour effectuer le décodage d'un flottant x , il suffit de repérer dans quelle succession d'intervalles celui-ci se trouve. À titre d'exemple, reprenons le tableau précédent et considérons le nombre $x = 0.123$.

Caractère	'a'	'b'	'c'	'd'	'e'
Fréquence	0.2	0.1	0.2	0.4	0.1
Intervalle	$[0, 0.2[$	$[0.2, 0.3[$	$[0.3, 0.5[$	$[0.5, 0.9[$	$[0.9, 1.0[$

Puisque x appartient à l'intervalle $[0, 0.2[$, le premier caractère est un 'a'. Puisque x appartient au sous-intervalle $[0.10, 0.18[$, le caractère suivant est un 'd', etc.

Q12.) Déterminer le caractère qui suit 'ad' dans la chaîne codée par $x=0.123$ en spécifiant le sous-intervalle qui a permis de décoder ce caractère.

```
a -> [0, 0.2[
d -> [0.1, 0.18[
b -> [0.12, 0.1375]
```

Q13.) Dans le cadre de l'exemple de cette partie, indiquer deux chaînes qui peuvent correspondre au flottant 0.2. Expliquer par une phrase ce qui est à l'origine de cette ambiguïté.

Les chaînes "b" et "ba" peuvent être confondues, en effet, on ne connaît pas la longueur de la chaîne, ou, plus précisément, le nombre de "a" à la fin de la chaîne. Une solution possible pour résoudre le problème précédent consiste à introduire un caractère nouveau signifiant la fin de la chaîne de caractères. Nous conviendrons de désigner ce caractère par '#'. Ce caractère se voit attribuer une plage non vide au voisinage de 0. Dans la suite, on suppose que la table des fréquences est adaptée de sorte à prendre en compte la présence de ce nouveau caractère.

On suppose disposer, en plus de la fonction `codeCar(car, g, d)` précédente, d'une fonction `decodeCar(x: float, g: float, d: float) -> str` qui détermine le caractère correspondant à la valeur x quand celle-ci est comprise dans la plage de $[g, d[$. Par exemple `decodeCar(0.123, 0, 1)` donne 'a' tandis que `decodeCar(0.123, 0, 0.2)` donne le caractère 'd'.

Q14.) Écrire une fonction `decodage(x: float) -> str` produisant la chaîne de caractères s déterminée par la valeur de x (avec le caractère '#' compris).

```
def decodage(x):
    s = ""
    c = None
    a = 0.
    b = 1.
    while c != "#":
        c = decodeCar(x, a, b)
        s += c
        a, b = codeCar(c, a, b)
    return s
```

II) Décodage du message reçu par Bob à l'aide de l'algorithme de Viterbi

II.1) Modélisation du canal de communication par un graphe

Dans cette partie, nous allons désormais considérer que le message compressé par Alice a été envoyé au travers d'un canal de communication. À cette fin, et indépendamment de la phase de compression étudiée dans la première partie, le message a subi une deuxième phase de transformation, dite d'encodage (cf Figure 1).

Le message envoyé sur le canal est une suite de symboles à valeurs dans un alphabet, noté Σ , comportant K symboles. Le choix d'un alphabet efficace n'est pas l'objet de notre étude et constitue un sujet à part entière. Suite au passage dans le canal de communication, le message envoyé subit

une altération de sorte que Bob reçoit une séquence de symboles de Σ qui ne correspond pas nécessairement à celle qui a été émise.

Dans cette partie, nous allons voir une approche permettant à Bob de décoder le message reçu et de potentiellement corriger quelques erreurs liées à la transmission du message et à la connaissance a priori de la propension du canal de communication à altérer les symboles du message lors de la transmission.

La modélisation proposée est la suivante :

- Bob observe une suite de N symboles $\text{obs}_0, \dots, \text{obs}_{N-1}$, que nous allons représenter par une liste Python `Obs = [obs0 , . . . obsN - 1 ]`,
- pour simplifier, on supposera que l'alphabet Σ est un ensemble de K entiers consécutifs commençant à 0, de sorte que $\Sigma = \llbracket 0, K - 1 \rrbracket$. Par exemple si $K = 3$ et $N = 8$, un message valide reçu par Bob pourrait être $[2, 0, 0, 2, 1, 1, 0, 0]$.
- chacun des symboles observés obs_t correspond à l'altération d'un symbole s_t envoyé par Alice. On note $[s_0, \dots, s_{N-1}]$ le message original ; pour reprendre l'exemple précédent, Alice pourrait avoir envoyé $[2, 0, 0, 2, 1, 1, 2, 0]$.
- on connaît, pour chaque paire $(i, j) \in \Sigma^2$, la probabilité $E_{i,j}$ que le canal altère le symbole j en un symbole i . On stocke ces probabilités dans une liste de listes `E` ; autrement dit, `E[i][j]` est la probabilité conditionnelle d'observer le symbole i sachant que le symbole j a été émis.

Ici (par exemple) on pourrait considérer :

$$E = \begin{pmatrix} 0.7 & 0.2 & 0.3 \\ 0.2 & 0.7 & 0.1 \\ 0.1 & 0.1 & 0.6 \end{pmatrix}$$

représentée par la liste de listes $E = [[0.7, 0.2, 0.3], [0.2, 0.7, 0.1], [0.1, 0.1, 0.6]]$ Le fait que $E[2][0]$ vaille 0.1 signifie donc que la probabilité que le symbole observé par Bob soit un 2 sachant qu'Alice a émis un 0 est de 0.1.

- on suppose également que le symbole courant s_t envoyé par Alice a une incidence sur le symbole suivant s_{t+1} qu'elle peut envoyer, au même titre que dans une langue comme le français, la probabilité d'observer un 't' dans un mot, n'est pas la même suivant que le caractère précédent est un 'e' ou un 'z'.

Ainsi pour chaque couple de symboles $(i, j) \in \Sigma^2$, on suppose que l'on connaît la probabilité d'émettre le symbole j à l'instant $t+1$ sachant que symbole i a été émis à l'instant t . On suppose également que cette probabilité ne dépend pas de t .

L'information concernant ces probabilités de transition d'un symbole à l'autre peut se stocker dans une matrice P de taille $K \times K$, que l'on représente informatiquement par une liste de listes P . Chaque entrée $P[i][j]$ donne la probabilité qu'Alice émette le symbole j à l'instant $t+1$ sachant qu'elle a émis le symbole i à l'instant t . En d'autres termes, $P[i][j] = P_{s_t=i}(s_{t+1}=j)$.

On prendra ici à titre d'exemple :

$$P = \begin{pmatrix} 0.3 & 0.2 & 0.5 \\ 0.4 & 0.4 & 0.2 \\ 0.2 & 0.3 & 0.5 \end{pmatrix}$$

représentée par la liste de listes $P = [[0.3, 0.2, 0.5], [0.4, 0.4, 0.2], [0.2, 0.3, 0.5]]$

Le fait que $P[2][0]$ vaille 0.2 signifie donc que la probabilité que le symbole envoyé par Alice à l'instant $t+1$ soit un 0 sachant que celui envoyé à l'instant t est un 2 vaut 0.2.

Nous allons désormais nous intéresser au problème du décodage : étant donné la liste $\text{Obs} = [\text{obs}_0, \dots, \text{obs}_{N-1}]$ des symboles observés par Bob, quelle séquence $\tilde{s}_0, \dots, \tilde{s}_{N-1}$ est la plus probable ?

En d'autres termes, $\tilde{s}_0, \dots, \tilde{s}_{N-1}$ est l'estimation la plus probable faite par Bob du message d'origine $s_0, \dots, s_{N-1}$, étant donné les observations $\text{obs}_0, \dots, \text{obs}_{N-1}$.

La modélisation précédente peut se représenter à l'aide d'un graphe défini comme suit (voir Figure 3 pour un exemple) :

- on crée un sommet $S_{i,j}$ pour chaque symbole possible $0 \leq i \leq K-1$ et chaque indice d'observation $0 \leq j \leq N-1$. Chaque couche verticale dans le graphe correspond à un caractère dans le message. Chaque strate horizontale correspond à un symbole.
- au niveau de la j -ème couche verticale, les sommets $S_{i,j}$ pour $j < N-1$ ont pour successeurs les états $S_{k,j+1}$ pour tous les symboles k possibles,
- par commodité, on ajoute un état source σ correspondant au début du message décodé et un état cible τ correspondant à la fin du message, ces états étant respectivement reliés à la première et la dernière couche,
- le décodage du message envoyé par Alice correspond à un chemin entre σ et τ dans ce graphe. A chaque sommet du chemin correspond une lettre décodée. Par exemple, le chemin passant par $S_{0,0}, S_{2,1}, S_{0,2}, S_{1,3}$, correspond au décodage de $[0, 2, 0, 1]$.

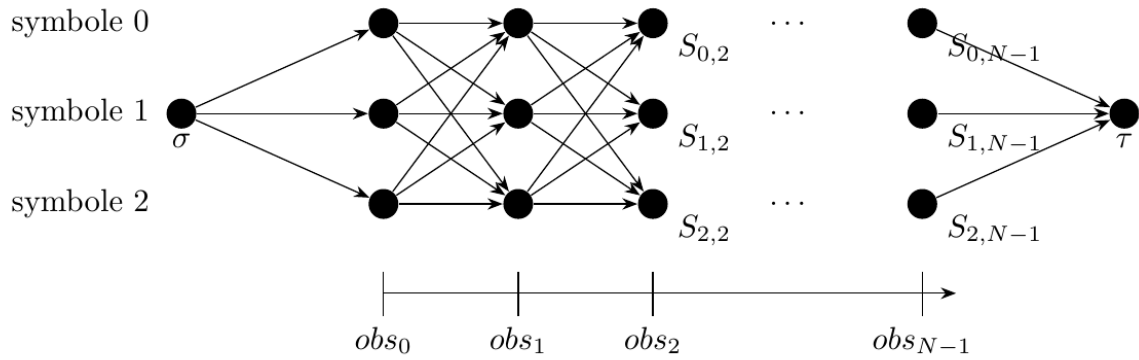


Figure 3: Illustration du modèle de décodage considéré ici.

À chacune des observations obs_i on peut faire correspondre K états. Chacun de ces états correspond à l'un des symboles potentiellement émis par Alice (ici $K = 3$). Les états sont organisés en N couches successives. Chaque état est noté $S_{i,j}$; j correspond à l'indice dans la suite des symboles émis et i au symbole correspondant. Les états de la couche j et les états de la couche $j + 1$ sont reliés par un arc de j vers $j + 1$. On ajoute également, par commodité, un état source σ et un état cible τ respectivement reliés à la première et à la dernière couche.

Q15.) En fonction de N et de K , donner le nombre de sommets et d'arcs du graphe illustré par la Figure 3. On ne comptera pas les sommets source σ et cible τ , ni les arcs partant du sommet source σ ni ceux arrivant à la cible τ .

On a K sommets pour chacune des N observations, pour un total de NK sommets.

Chaque groupe de K sommet est entièrement relié au groupe de K sommet suivants, ce qui donne K^2 arrêtes.

Cela arrive $N - 1$ fois.

Il y a donc $(N - 1)K^2$ arrêtes.

On choisit désormais de pondérer chaque arc par la probabilité de transiter par cet arc. Autrement dit :

- les arcs issus de la source σ vers $S_{i,0}$ sont pondérés par $E_{obs_0,i}$ la probabilité d'observer le symbole obs_0 sachant que le symbole i a été émis par Alice ;
- les arcs arrivant à la cible τ sont pondérés par 1 (en fin de message, on transite forcément vers l'état final),
- les arcs internes entre $S_{i,j}$ et $S_{k,j+1}$ sont pondérés par la probabilité $E_{obs_{j+1},k} P_{i,k}$.

La probabilité d'un chemin $\sigma S_{i_0,0} S_{i_1,1} \dots S_{i_{N-1},N-1} \tau$ entre σ et τ est le **produit** des probabilités des arcs qui le composent.

L'objectif va être de trouver le chemin de **probabilité maximale** dans ce graphe entre le sommet source σ et le sommet cible τ .

Q16.) On suppose que Bob a observé la séquence $[2,0]$. En utilisant les matrices E et P données dans l'énoncé (avec $K = 3$), construire le graphe pondéré associé à ce message de longueur $N = 2$. Les arcs entre les sommets devront être pondérés par les probabilités correspondantes.

Q17) On revient dans le cas général, N et K sont désormais quelconques. Indiquer combien il existe de chemins entre σ et τ (un ordre de grandeur utilisant la notation O est accepté). Préciser si un algorithme d'exploration exhaustive est envisageable dans ce cas.

A chaque étape (sauf la dernière), on a le choix entre K sommets. Il y a N étapes en comptant le passage de σ au reste de graphe.

On a donc K^N chemins possibles.

Ce n'est pas du tout réalisable ! Si on a 10 lettres et un message de 20 caractères, cela donne 10^{20} opérations, bien plus que ce qu'un ordinateur peut calculer en un temps raisonnable.

II.2) Stratégie gloutonne

Pour pouvoir implémenter correctement la recherche du chemin de probabilité maximale, il est utile de disposer d'une fonction auxiliaire qui sera utilisée dès que nécessaire.

Q18) Pour une liste liste, on appelle argument du maximum et on note argMax tout indice i tel que $\text{liste}[i]$ soit maximal. Proposer une fonction `maximumListe(liste:[float]) -> (float, int)` qui prend en entrée une liste de nombres et qui renvoie la valeur du maximum de la liste ainsi que le plus petit argument du maximum, i.e. le premier indice auquel cette valeur maximale apparaît.

```
def maximumListe(liste):  
    arg = 0  
    for i in range(1, len(liste)):  
        if liste[i] > liste[arg]:  
            arg = i  
    return liste[arg], arg
```

On souhaite appliquer un algorithme glouton pour trouver le chemin de probabilité maximale entre le sommet source σ et le sommet cible τ . On rappelle qu'un algorithme glouton cherche, à chaque étape, à faire le choix localement optimal. Ici, si à une étape on se retrouve au sommet $S_{i,j}$, il s'agit de choisir l'arc de plus forte probabilité partant de ce sommet.

Dans un premier temps on écrit une fonction `initialiserGlouton(Observe:[int], E:[float], K:int) -> int` qui permet d'initialiser l'algorithme glouton en trouvant le sommet le plus probable parmi $S_{i,0}$ pour i variant entre 0 et $K - 1$. Pour cela il faut regarder la colonne `Observe[0]` de `E` et relever l'indice de la plus grande valeur.

```
def initialiserGlouton(Observe, E, K):  
    probasInitiales = [E[Observe[0]][i] for i in range(K)]  
    s, symbole = maximumListe(probasInitiales)  
    return symbole
```

Q19) Proposer une fonction `glouton(Observe:[int], P:[float], E:[float], K:int, N:int) -> [int]` qui renvoie la liste d'états obtenue par l'approche gloutonne. Même si cela n'est pas nécessaire, K , N seront des arguments de cette fonction.

```
def glouton(Observe, P, E, K, N):  
    S = [initialiserGlouton(Observe, E, K)]  
    for i in range(1, N):  
        probas = [E[Observe[i]][k] * P[S[i-1]][k] for k in range(K)]  
        val, arg = maximumListe(probas)
```

```

    S.append(arg)
return S

```

Q20.) En fonction de K et de N , quelle est, en ordre de grandeur, la complexité temporelle asymptotique de l'approche gloutonne ?

A chaque étape, le calcul de la proba max est en $O(K)$. Donc au total, l'approche gloutonne est en $O(NK)$

Q21.) Indiquer le chemin renvoyé par l'algorithme glouton appliqué à la Figure 4. Conclure quant à l'optimalité de l'approche.

Le glouton emprunte $\theta - 1$ pour une probabilité totale de 0.3.

Or, le chemin $1 - \theta$ a une probabilité de 0.36. Donc le glouton n'est pas optimal.

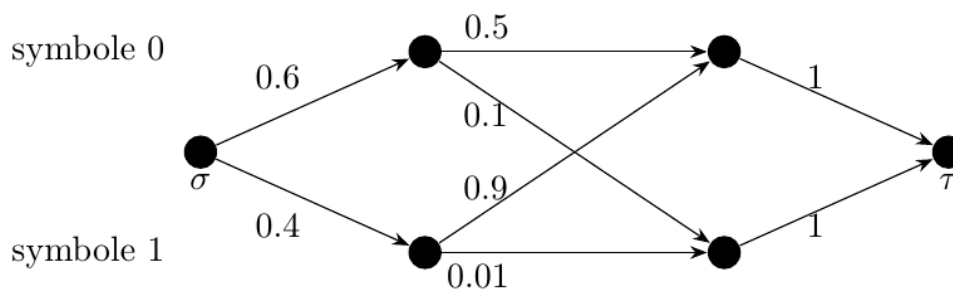


Figure 4: Que donne l'algorithme glouton ici ?